

JS : Les Librairies



ADRAR / PÔLE NUMERIQUE

- > **INFRASTRUCTURES** SYSTÈMES & RÉSEAUX
- > **CYBERSÉCURITÉ** INFRASTRUCTURES & APPLICATIONS
- > **DEVOPS / SCRIPTING** & AUTOMATISATION
- > **DEVELOPPEMENT** WEB & MOBILE
- > TRANSFORMATION NUMERIQUE DES ENTREPRISES

www.adrar-numerique.com

Sommaire

- 01** Pourquoi une bibliothèque ?
- 02** Installer Node.js & npm
- 03** Créer un projet npm
- 04** Structure de dossier propre
- 05** Installer DOMPurify
- 06** DOMPurify côté serveur
- 07** Importer la bibliothèque
- 08** Centraliser dans une fonction
- 09** Utiliser dans l'application
- 10** Vérifier le résultat
- 11** Bonnes pratiques & le cycle complet
- 12** Bibliothèque vs API + Bonus

Pourquoi utiliser une bibliothèque ?

Un projet a régulièrement besoin de fonctionnalités qui existent déjà : nettoyer du HTML, afficher des notifications, créer des animations...

Développer nous-mêmes

Réécrire depuis zéro une fonctionnalité qui existe déjà ailleurs.

Réutiliser le travail déjà fait

Utiliser une bibliothèque déjà écrite et testée - le choix pro.

Une bibliothèque, c'est du code écrit par d'autres développeurs, que nous ajoutons à notre projet pour ne pas avoir à l'écrire nous-mêmes.

Gagner du temps

Utiliser du code déjà testé

Se concentrer sur son application

npm : l'outil pour installer des bibliothèques

Pour installer des bibliothèques modernes, il faut npm (Node Package Manager). npm ne s'installe pas seul : il est fourni automatiquement avec Node.js.

Windows

- Touches Windows + R
- Taper : cmd
- Valider avec Entrée

macOS

- Ouvrir Terminal
- Depuis Applications
- ...ou via Spotlight

Linux

- Ouvrir Terminal
- Depuis le menu applications

On n'utilise pas Node.js directement dans ce cours, on en a besoin uniquement pour npm.

Vérifier l'installation

```
$ node -v  
v22.15.0
```

```
$ npm -v  
10.9.2
```

Si les deux commandes affichent un numéro de version, on peut continuer. Sinon, installer Node.js depuis nodejs.org avant de poursuivre.

On n'utilise pas Node.js directement dans ce cours, on en a besoin uniquement pour npm.

Créer le projet

```
$ mkdir mon-projet  
  
$ cd mon-projet  
  
$ npm init -y
```

- Cette commande crée automatiquement un fichier **package.json**
- Sans ce fichier, npm ne sait pas dans quel projet enregistrer une bibliothèque

```
{  
  
  "name": "mon-projet",  
  "version": "1.0.0"  
}
```

package.json : la carte d'identité du projet

```
{  
  "name": "mon-projet",  
  "version": "1.0.0",  
  "dependencies": {  
    "dompurify": "^3.2.7"  
  }  
}
```

- Nom et version du projet
- Liste des bibliothèques utilisées (dependencies)
- Créé automatiquement par npm init -y

Pourquoi avant d'installer une bibliothèque ?

npm a besoin de savoir dans quel projet enregistrer la bibliothèque installée.

Un collègue récupère le projet ?

Il récupère juste le dossier, tape npm install, et npm retélécharge tout seul.

Une structure de dossier propre

```
mon-projet/  
|-- node_modules/      (cree par npm, on n'y touche jamais)  
|-- public/  
|   |-- index.html  
|   +-- css/style.css  
|-- src/  
|   |-- js/  
|   |   |-- app.js      -> point d'entree  
|   |   |-- sanitizer.js -> nettoyage (securite)  
|   |   |-- validation.js -> entrees utilisateur  
|   |   |-- api.js       -> appels API / serveur  
|   |   +-- ui.js        -> interface  
|   +-- assets/  
|-- package.json  
|-- package-lock.json  
+-- README.md
```

Chaque fichier a une seule responsabilité : plus facile à maintenir, à tester, et à faire relire par quelqu'un d'autre.

DOMPurify — le problème à résoudre

Un utilisateur peut saisir du HTML dans un champ texte (volontairement ou non 🤖).

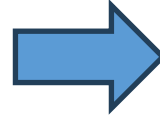
```
// Ce que l'utilisateur saisit :  
<script>alert("Hack");</script>  
  
// Si on affiche directement ce texte :  
element.innerHTML = userInput;
```

Faible XSS (Cross-Site Scripting) - le navigateur exécute le script inséré.

DOMPurify nettoie le HTML avant qu'on l'affiche, pour éviter ce problème.

Installer DOMPurify

```
$ npm install dompurify
```



- 1 Télécharge DOMPurify
- 2 Range son code dans node_modules/
- 3 Ajoute la ligne dans package.json

```
    "dependencies": {  
      "dompurify": "^3.2.7"  
    }
```

Attention : la bibliothèque est téléchargée, mais pas encore utilisée.

DOMPurify côté serveur (Node.js)

DOMPurify a besoin d'un DOM (la structure d'une page web) pour fonctionner.

Navigateur

- Le DOM existe déjà
- DOMPurify fonctionne directement, rien à installer de plus

Node.js

- Aucun DOM par défaut
- Il faut une bibliothèque qui simule un DOM

```
$ npm install jsdom
```

Importer la bibliothèque

src/js/sanitizer.js

```
import DOMPurify from "dompurify";
```

Cette ligne signifie : va chercher le code de DOMPurify dans node_modules et rends-le disponible dans ce fichier.



Centraliser l'utilisation dans une fonction

src/js/sanitizer.js

```
import DOMPurify from "../../node_modules/dompurify/dist/purify.es.mjs";

export function nettoyer(texte) {
  return DOMPurify.sanitize(texte);
}
```

*Attention, le chemin RELATIF vers DOMPurify change en fonction d'où se trouve votre fichier sanitizer.js
Plutôt que d'appeler DOMPurify.sanitize() à plusieurs endroits du projet, on crée une seule fonction que tout le projet réutilise.*

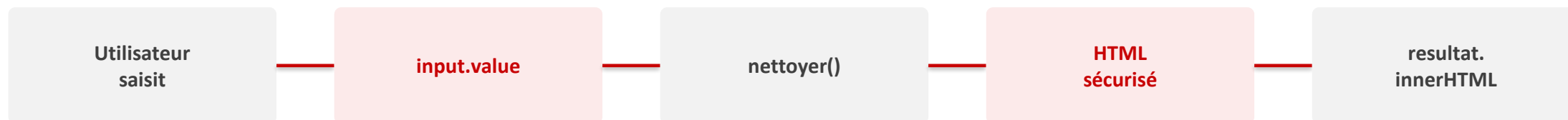
Utiliser cette fonction dans l'application

src/js/app.js

```
import { nettoyer } from "../sanitizer.js";

const input = document.querySelector("#message");
const resultat = document.querySelector("#resultat");

input.addEventListener("input", () => {
  const propre = nettoyer(input.value);
  resultat.innerHTML = propre;
});
```



On n'écrit jamais `input.value` → `innerHTML` directement, sans passer par `nettoyer()`.

Vérifier le résultat

Ce que l'utilisateur écrit	Ce que DOMPurify renvoie
Bonjour	Bonjour - aucun changement
<code>Bonjour</code>	HTML autorisé conservé
<code><script>alert("Hack")</script></code>	Le script est supprimé
<code></code>	Attribut dangereux supprimé

DOMPurify conserve le HTML autorisé mais supprime tout ce qui est dangereux (scripts, attributs comme onerror).

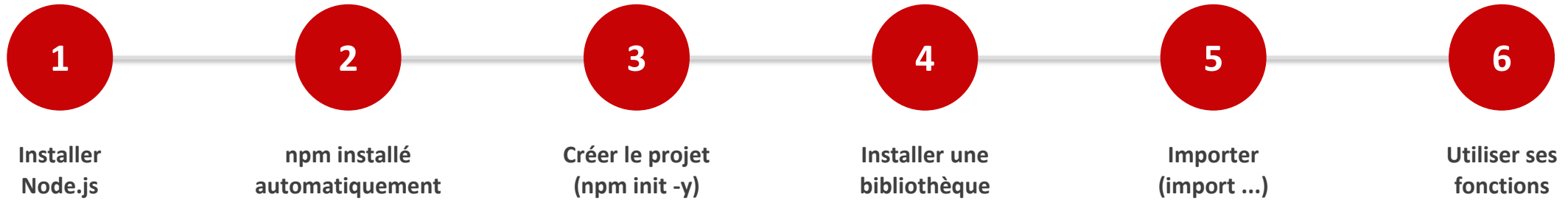
Bonnes pratiques à retenir

- Toujours installer une bibliothèque avec **npm install**, jamais en copiant le code à la main
- Créer un fichier dédié (**sanitizer.js**) plutôt que d'appeler la bibliothèque partout dans le projet
- Nettoyer toute donnée saisie par un utilisateur avant de l'insérer avec **innerHTML**
- Pour du texte simple sans HTML, utiliser **textContent** : pas besoin de nettoyage

```
// Affichage de texte simple, sans risque :  
resultat.textContent = input.value;  
  
// Affichage de HTML volontairement autorisé, nettoyé au préalable :  
resultat.innerHTML = nettoyer(input.value);
```

Le cycle complet

Valable pour (presque) toutes les bibliothèques JavaScript modernes.



Bibliothèque vs API — ne pas confondre

	Bibliothèque	API
Quoi	Du code ajouté à notre projet	Un service distant auquel on envoie une requête
Où s'exécute-t-elle	Localement, dans notre projet	Sur un serveur externe
Exemple	DOMPurify, party.js	API Giphy, API Tenor
Comment on l'utilise	npm install puis import	fetch() vers une URL

Une bibliothèque = code local ajouté au projet (npm install). Une API = service distant qu'on interroge (fetch()).

Bonus : party.js — des confettis en 3 lignes

Pour un effet visuel immédiat et parlant : party.js déclenche des confettis.

```
$ npm install party-js  
  
import party from "party-js";  
  
bouton.addEventListener("click", () => party.confetti(bouton));
```

1

npm install
party-js

2

import
party-js

3

party.confetti
(bouton)

4

Résultat visuel
immédiat

Ce qu'il faut retenir

- Une bibliothèque = **du code déjà écrit et testé**, à ajouter à son projet
- Le cycle **npm install** → **import** → **utilisation** est (presque) toujours le même
- Toujours nettoyer les entrées utilisateur avant un **innerHTML** (DOMPurify)
- Ne jamais copier le code d'une bibliothèque à la main : toujours **npm install**

Une fois ce cycle compris, on peut intégrer n'importe quelle bibliothèque JavaScript moderne.